

Advantages and Disadvantages of Arrays in C#

In this article, I am going to discuss the **Advantages and Disadvantages of Arrays in C#** with examples. This is one of the most frequently asked C# interview questions. Before proceeding to this article, I strongly recommended you read the following two articles.

1. [Arrays in C#](#) – Here we discussed the basic concepts of arrays along with a one-dimensional array with examples.
2. [Two-Dimensional Arrays in C#](#) – Here we discussed the two-dimensional arrays along with jagged arrays in C# with examples.

Advantages of using an Array in C#:

The advantages of using an array in C# are as follows:

1. It is used to store similar types of multiple data items using a single name.
2. We can use arrays to implement other data structures such as linked lists, trees, graphs, stacks, queues, etc.
3. The two-dimensional arrays in C# are used to represent matrices.
4. The Arrays in C# are strongly typed. That means they are used to store similar types of multiple data items using a single name.

As the arrays are strongly typed so we are getting two advantages. First, the performance of the application will be much better because boxing and unboxing will not happen. Secondly, runtime errors will be prevented because of a type mismatch. In this case, at compile time it will give you the error if there is a type mismatch.

Example: Arrays give us compilation error when type mismatch

In the following example, we create an integer array with the name `numberArray`. As it is an integer array so we can store only the integer values within the array. As you can see when we try to store a string value within the array, immediately it gives us a compiler error saying “**cannot implicitly convert a string to an integer**”. This is the reason why we call the arrays in C# strongly typed.

```

class Program
{
    static void Main(string[] args)
    {
        //Creating an Integer Array to store 3 integer numbers
        int[] numberArray = new int[3];

        //Storing Integer Number is fine
        numberArray[0] = 10;

        //Trying to store string value
        numberArray[1] = "ABC";
    }
}

```

 **class System.String**
Represents text as a sequence of UTF-16 code units.
Cannot implicitly convert type 'string' to 'int'

You can get type mismatches and runtime errors with collection classes like **ArrayList**, **Queue**, **Stack**, etc which are present in the **System.Collections** namespace. We will discuss collections in detail in our upcoming articles. But in this article, to make you understand the type mismatch let us create an example using the ArrayList which is a collection in C#.

Type Mismatch Example using ArrayList Collection in C#:

In the following example, we create one collection using the ArrayList collection class with the name numberArray. The collection classes that are present in the System.Collections namespace such as **ArrayList** is loosely typed. Loosely typed means, you can store any type of data in that collection. The **ArrayList** in C# is operated on object data type, which makes the ArrayList loosely typed. So, you will not get any compile-time error, but when you run the application, you will get a runtime error.

```

using System;
using System.Collections;
namespace ArrayDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Creating a Collection using Array List
            ArrayList numberArray = new ArrayList();
            numberArray.Add(10);
            numberArray.Add(200);
            //No CompileTime Error
            numberArray.Add("Pranaya");
            //We Get Runtime Error, when we access the 3rd Element

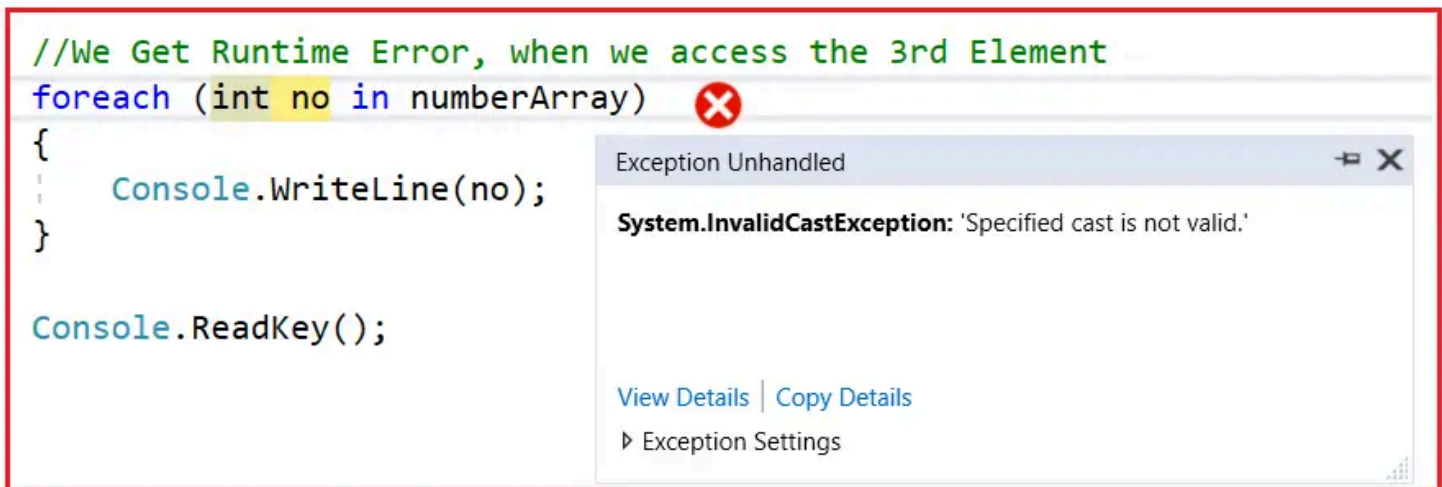
```

```

        foreach (int no in numberArray)
        {
            Console.WriteLine(no);
        }
        Console.ReadKey();
    }
}

```

When you run the application, at runtime you will get the following exception. Here, it is trying to convert and store the string value into an integer variable and the casting is not valid, and hence it is throwing an exception saying that the Specified cast is not valid. If we use Array instead of ArrayList, then this kind of Exception will never get.



Disadvantages of using Arrays in C#:

1. The array size is fixed. So, we should know in advance how many elements are going to be stored in the array. Once the array is created, then we can never increase or decrease the size of an array. If you want then we can do it manually by creating a new array and copying the old array elements into the new array.
2. As the array size is fixed, if we allocate more memory than the requirement then the extra memory will be wasted. On the other hand, if we allocate less memory than the requirement, then it will create a problem.
3. We can never insert an element into the middle of an array. It is also not possible to delete or remove elements from the middle of an array.

To overcome the above problems Collections are introduced in C#. And from the next and a few upcoming articles, we are going to discuss the Collection Framework in detail.

In the article, we are going to start **Collections in C#**. Here, in this article, I try to explain the **Advantages and Disadvantages of Arrays in C#** with examples. I hope this article will help you with your needs. I would like to have your feedback. Please post your feedback, question, or comments about this article.